Security Audit Report - سوق الكل (swkalkillcom)

Date: March 13, 2026
Repository: swkalkillcom
Type: Capacitor + React + TypeScript Web/Mobile App
Audit Level: Full Repository Security Check

Executive Summary

The application has **28 vulnerabilities** (19 HIGH, 7 MODERATE, 2 LOW severity) primarily in development dependencies. The most critical issues are in build tools and CLI utilities that could impact development/CI/CD pipelines. Additionally, there are several security best practices that should be implemented for production safety.

Risk Level: MEDIUM-HIGH (Most risks are in dev dependencies, but some code-level issues need addressing)

🔴 CRITICAL FINDINGS

1. **Exposed Secrets in Repository** (HIGH SEVERITY)

Status: ⚠ CRITICAL - Action Required

The `.env` file is tracked in git with Supabase credentials exposed:

```
File: ./.env
VITE_SUPABASE_PROJECT_ID="dhkdgdehcwcglhntsegs"
VITE_SUPABASE_PUBLISHABLE_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
VITE_SUPABASE_URL="https://dhkdgdehcwcglhntsegs.supabase.co"
```

PROF

Risk: While these are "publishable" keys (meant for frontend), they should never be committed. The project ID exposes infrastructure information.

Recommended Actions:

1. Immediately remove `.env` from git history:

```
git rm --cached .env
git commit -m "Remove .env file from tracking"
```

2. Create `.env.example` template without values
3. Ensure `.gitignore` includes `.env` (already present - ✅ good)
4. Add pre-commit hook to prevent accidental commits

5. **CRITICAL:** Regenerate Supabase keys if this repo is public

---

🔴 HIGH SEVERITY VULNERABILITIES (19 total)

1. **Dependency Vulnerabilities - Build Tools**

a) minimatch - ReDoS Vulnerabilities (MULTIPLE HIGH)

- **Affected Packages:** eslint, tailwindcss, @capacitor/cli, vite-plugin-pwa
- **CVE:** GHSA-3ppc-4f35-3m26, GHSA-7r86-cg39-jmmj, GHSA-23c5-xmqv-rm74
- **Impact:** Denial of Service (DoS) via Regular Expression
- **Action:** `bun update minimatch`

b) tar - File Extraction Vulnerabilities (3 HIGH)

- **Affected Package:** @capacitor/cli › tar
- **CVEs:** GHSA-83g3-92jg-28cx, GHSA-qffp-2rhf-9h96, GHSA-9ppj-qmqm-q256
- **Impact:** Arbitrary file read/write, symlink path traversal
- **Action:** Update to tar >= 7.5.8

c) rollup - Arbitrary File Write (2 HIGH)

- **Affected Package:** vite › rollup, vite-plugin-pwa › workbox-build › rollup
- **CVE:** GHSA-mw96-cpmx-2vgc
- **Impact:** Path traversal vulnerability in Rollup 4
- **Action:** `bun update rollup` to 4.59.0+

d) @remix-run/router - XSS via Open Redirects (HIGH)

- **Affected Package:** react-router-dom › react-router › @remix-run/router
- **CVE:** GHSA-2w69-qvjg-hvjx
- **Impact:** Cross-Site Scripting (XSS) vulnerability
- **Current Version:** <= 1.23.1
- **Action:** Update react-router-dom to 6.30.2+ immediately
- **Risk Level:** HIGH - This affects navigation security

e) glob - Command Injection (HIGH)

- **Affected Package:** @capacitor/cli › rimraf › glob
- **CVE:** GHSA-5j98-mcp5-4vw2
- **Impact:** Shell command injection via -c/--cmd flag
- **Action:** Update glob to 10.5.0+

2. **React Router Security Issue** (HIGH)

File: src/hooks/useAuth.tsx, src/hooks/useAuth.tsx

PROF

Issue: Hardcoded redirects using `window.location.origin`

```
emailRedirectTo: window.location.origin,
redirectTo: `${window.location.origin}/settings`,
```

Risk: While not inherently unsafe with origin validation, this could be vulnerable to open redirect attacks if not properly validated by Supabase backend.

Recommendation: Explicitly define safe redirect URLs in config instead of relying on location.origin.

🟠 MODERATE SEVERITY VULNERABILITIES (7 total)

1. **esbuild - Development Server Security** (MODERATE)

- **Affected Packages:** lovable-tagger › esbuild, vite › esbuild
- **CVE:** GHSA-67mh-4wv8-2f99
- **Impact:** Dev server allows arbitrary requests and reads responses
- **Action:** Upgrade esbuild to 0.24.3+
- **Note:** Mostly affects development; less critical for production

2. **Vite File Serving Bypasses** (3 MODERATE)

- **Affected Package:** vite <= 5.4.19
- **CVEs:** GHSA-g4jq-h2w9-997c, GHSA-93m4-6634-74q7, GHSA-jqfw-vq24-v9c3
- **Impact:** Bypass of `server.fs` restrictions, file access issues
- **Current Version:** 7.3.1
- **Status:** Already updated ✅
- **Recommendation:** Maintain current version

3. **ajv - ReDoS Vulnerability** (MODERATE)

PROF

- **Affected Package:** eslint › @eslint/eslintrc › ajv
- **CVE:** GHSA-2g4f-4pwh-qvx6
- **Impact:** Regex Denial of Service
- **Action:** `bun update ajv`

4. **lodash - Prototype Pollution** (MODERATE)

- **Affected Package:** recharts › lodash, vite-plugin-pwa › workbox-build › lodash
- **CVE:** GHSA-xxjr-mmjv-4gpg
- **Impact:** Prototype pollution in _.unset and _.omit
- **Action:** `bun update lodash`

5. **js-yaml - Prototype Pollution** (MODERATE)

- **Affected Package:** eslint › @eslint/eslintrc › js-yaml

- **CVE:** GHSA-mh29-5h37-fv8m
- **Impact:** Prototype pollution in merge
- **Action:** Update js-yaml to 4.1.1+

6. **React Router - Unexpected Redirects** (MODERATE)

- **Affected Package:** react-router >= 6.0.0 < 6.30.2
- **CVE:** GHSA-9jcx-v3wj-wh4m
- **Impact:** Untrusted path redirects
- **Action:** Ensure react-router-dom >= 6.30.2

🟢 LOW SEVERITY VULNERABILITIES (2 total)

1. **Vite Development Server Issues** (2 LOW)

- **Affected Packages:** vite <= 5.4.19
- **CVEs:** GHSA-g4jq-h2w9-997c, GHSA-jqfw-vq24-v9c3
- **Impact:** Minor file serving edge cases
- **Status:** Already patched in version 7.3.1 ✅

🔐 CODE-LEVEL SECURITY ISSUES

1. **XSS Risk - dangerouslySetInnerHTML** (MEDIUM)

File: src/components/ui/chart.tsx

```
dangerouslySetInnerHTML={{
  __html: Object.entries(THEMES)
    .map(([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig...}
```

PROF

Status: ✅ SAFE - This generates only static CSS variables, no user input is interpolated.

Recommendation: Add a comment explaining why this is safe for future maintainers.

---

2. **Supabase Query Safety** (MEDIUM CONCERN)

Files: Multiple pages and hooks using Supabase

All queries use parameterized queries via Supabase SDK:

```
.from("listings")
.select("*")
.eq("id", listingId)
```

Status: ✅ SAFE - Supabase SDK prevents SQL injection through its typed query builder.

---

3. **localStorage Usage** (LOW CONCERN)

Files:

- src/pages/Settings.tsx
- src/contexts/ThemeContext.tsx
- src/contexts/LanguageContext.tsx

```
localStorage.getItem("notification_sound")
localStorage.setItem("theme", theme)
```

Status: ✅ SAFE - Only storing non-sensitive user preferences.

Note: Do NOT store authentication tokens in localStorage. Ensure Supabase client configuration uses proper auth storage.

---

4. **Authentication Configuration** (GOOD)

File: src/integrations/supabase/client.ts

```
auth: {
  storage: localStorage,
  persistSession: true,
  autoRefreshToken: true,
}
```

PROF

Status: ✅ GOOD - Using Supabase's built-in session management.

Recommendation: Consider adding HttpOnly cookie support for sensitive environments.

---

5. **API Function Calls Security** (GOOD)

File: src/pages/DeleteData.tsx

The delete account function properly:

- ✅ Validates authorization header
- ✅ Uses bearer token authentication
- ✅ Includes API key header
- ✅ Calls Supabase edge functions (server-side execution)

Status: ✅ SECURE

⚠ ADDITIONAL SECURITY RECOMMENDATIONS

1. **Content Security Policy (CSP)**

Status: × NOT IMPLEMENTED

Recommendation: Add CSP headers to prevent XSS attacks:

```
<!-- In index.html or configure in server -->
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self'; script-src 'self' 'wasm-unsafe-eval';
               connect-src 'self'
https://dhkdgdehcwcglhntsegs.supabase.co;">
```

2. **Environment Configuration**

Status: ⚠ PARTIALLY GOOD

- ✅ Supabase keys properly prefixed with `VITE_` (frontend-only keys)
- ✅ `.env` in .gitignore
- × `.env` file still committed to repo with live credentials

3. **CORS Configuration**

File: Supabase edge function supabase/functions/delete-account/index.ts

```
const corsHeaders = {
  "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey,
content-type",
};
```

Issue: 🔴 CORS set to allow all origins (*)

Recommendation: Restrict to specific domains:

```
const corsHeaders = {
  "Access-Control-Allow-Origin": "https://yourdomain.com",
  "Access-Control-Allow-Methods": "POST, OPTIONS",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey,
content-type",
};
```

PROF

4. **Input Validation**

Files: Various forms and input fields

Status: ✅ PARTIAL - Uses Zod for schema validation

Recommendation: Ensure all user inputs are validated:

- Phone numbers
- Emails
- Search queries
- File uploads

5. **Authentication Redirect Validation**

Issue: Redirect URLs use `window.location.origin` without explicit validation

Fix:

```
const ALLOWED_REDIRECTS = ['/settings', '/account', '/'];
const validRedirect = ALLOWED_REDIRECTS.includes(path) ? path : '/';
redirectTo: `${window.location.origin}${validRedirect}`
```

🔧 IMMEDIATE ACTION ITEMS

Priority 1 (CRITICAL - Do First)

- [ ] Remove `.env` from git history: `git rm --cached .env && git commit`
- [ ] Create `.env.example` file
- [ ] Update react-router-dom to >= 6.30.2
- [ ] Regenerate Supabase credentials if repo is public

PROF

Priority 2 (HIGH - Do Soon)

- [ ] Run `bun update` to update all dependencies
- [ ] Update CORS headers in edge functions
- [ ] Add explicit redirect URL validation
- [ ] Implement Content-Security-Policy headers

Priority 3 (MEDIUM - Do This Sprint)

- [ ] Add environment-specific configurations
- [ ] Implement request rate limiting
- [ ] Add security headers middleware
- [ ] Set up automated dependency scanning (e.g., Dependabot)

🏷️ DEPENDENCY UPDATE COMMANDS

```
# Update all dependencies
bun update

# Or update specific vulnerable packages
bun add @remix-run/router@latest
bun add react-router-dom@latest
bun add minimatch@latest
bun add tar@latest
bun add rollup@latest
bun add lodash@latest
```

🛡️ SECURITY CHECKLIST

- ☐ Regenerate all API keys (if repo was public)
- ☐ Review Supabase RLS (Row Level Security) policies
- ☐ Enable Supabase MFA enforcement
- ☐ Implement rate limiting on edge functions
- ☐ Add security headers (X-Content-Type-Options, X-Frame-Options, etc.)
- ☐ Enable HTTPS only for all connections
- ☐ Set up CORS restrictions
- ☐ Audit database schema for permission leaks
- ☐ Implement request signing/validation
- ☐ Set up security monitoring/logging

📊 Vulnerability Summary

Severity	Count	Status
Critical	1	🔴 Action Required
High	19	⚫ Update Dependencies
Moderate	7	🔘 Update Dependencies
Low	2	✅ Already Patched
Code Issues	2	✅ Safe

✅ POSITIVE SECURITY FINDINGS

1. ✅ Supabase authentication properly configured
2. ✅ Uses parameterized queries (no SQL injection risk)
3. ✅ Proper authorization token handling
4. ✅ Secure Supabase edge functions

PROF

5. ✅ No sensitive data in localStorage
6. ✅ TypeScript for type safety
7. ✅ .env properly in .gitignore (though file is committed)
8. ✅ ESLint configured for code quality

🔗 References

- Supabase Security Best Practices
- OWASP Top 10
- Node.js Security Best Practices
- React Security Best Practices

Report Generated: March 13, 2026
Next Review: After all Priority 1 & 2 items are completed